

A COURSE COMPANION



NumPy Book

Lessons & Explanations

A lesson-by-lesson walkthrough of **NumPy**, built from the YouTube course by **JoeTech**.

What's inside

01	The ndarray — NumPy's Core Object	3
	<i>Sample lesson — JoeTech, NumPy Course, Video 1 (placeholder for design review)</i>	

The ndarray — NumPy's Core Object

Sample lesson — JoeTech, NumPy Course, Video 1 (placeholder for design review)

If you remember one idea from this lesson, make it this one: **NumPy exists because Python lists are slow for numerical work, and the ndarray is the object that fixes that.** A Python list can hold anything — an int, a string, another list — and Python pays for that flexibility by storing each element as a separate object scattered in memory. An ndarray gives that up on purpose: every element has the same type and sits in one continuous block of memory, which is what lets NumPy hand the actual math off to fast, compiled C code instead of looping in Python.

What actually is an ndarray?

Think of an ndarray as three things bundled together: a block of raw memory, a **shape** that says how to read that memory as rows/columns/etc., and a **dtype** that says how many bytes each element takes and how to interpret them. Once you view it that way, a lot of "weird" NumPy behavior stops being weird — reshaping is just relabeling the same memory, and a slice is just a different way of walking over it.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[1, 2, 3], [4, 5, 6]])

print(a.shape)    # (3,)      → 1 axis, 3 elements
print(b.shape)    # (2, 3)    → 2 axes: 2 rows, 3 columns
print(b.dtype)    # int64     → every element is a 64-bit integer
```

You rarely build arrays element by element. NumPy ships constructors for the shapes you need most often:

FUNCTION	PRODUCES
<code>np.zeros((r, c))</code>	array filled with <code>0.0</code>

FUNCTION	PRODUCES
<code>np.ones((r, c))</code>	array filled with <code>1.0</code>
<code>np.arange(start, stop, step)</code>	evenly spaced values, like <code>range()</code>
<code>np.linspace(start, stop, n)</code>	<code>n</code> evenly spaced values, inclusive of both ends
<code>np.eye(n)</code>	an <code>n x n</code> identity matrix

WATCH OUT

A NumPy array's `dtype` is fixed at creation. Assign a float into an integer array and NumPy quietly truncates it instead of raising an error. Coming from plain Python lists, this is one of the easiest bugs to write without noticing:

```
ints = np.array([1, 2, 3])
ints[0] = 9.9
print(ints)    # [9 2 3] -- silently truncated, no warning
```

Shape, axes, and reshaping

Every ndarray carries a `.shape` tuple describing its size along each axis, and a `.ndim` telling you how many axes there are. `reshape()` does **not** copy data — it changes how the same underlying block of memory is *viewed*:

```
c = np.arange(12)
grid = c.reshape(3, 4)

print(grid)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

print(grid.T)           # transpose: swap the two axes
print(grid.flatten())    # back to 1-D, but this one is a copy
```

Because `reshape` hands back a view whenever it can, modifying `grid` after this point can also change `c` — they point at the same memory. This view-vs-copy distinction is the single idea that keeps coming back throughout the course: NumPy operations are

fast largely *because* they avoid copying, so it always pays to know which ones share memory and which ones don't.

Indexing and slicing

Indexing an ndarray reads like indexing a list, but it extends naturally to multiple dimensions with a comma inside the brackets instead of chained `[]`:

```
grid[0, 1]      # row 0, column 1      → 1
grid[:, 0]      # every row, column 0  → [0, 4, 8]
grid[1:, :2]    # rows 1 onward, first two columns
```

TIP

A slice of an ndarray is a **view**, not a copy — same rule as reshape. If you need an independent array, call `.copy()` explicitly: `sub = grid[1:, :2].copy()`.

KEY TAKEAWAYS

- An ndarray trades Python's per-element flexibility for one fixed `dtype` and one contiguous block of memory — that trade is the whole reason NumPy is fast.
- `reshape()` and basic slicing return **views**, not copies. Two arrays can silently share the same underlying data.
- Assigning a value of the "wrong" type into an array truncates or converts it silently — there's no automatic error to catch this for you.
- Everything later in the course — broadcasting, vectorised math, aggregation along an axis — builds directly on `.shape`, `.dtype`, and this view/copy distinction.